

Control Transfer Statements

break →

{ while condition :
 if condition_to_break :
 break

```
count = 0  
while True:  
    print("Inside the while loop")  
    count += 1  
    if count == 5:  
        break
```

Count = 0

Inside the while loop ✓

Count = 1

Inside the while loop ✓

Count = 2

Inside the while loop ✓

Count = 3

Inside the while loop ✓

Count = 4

Inside the while loop ✓

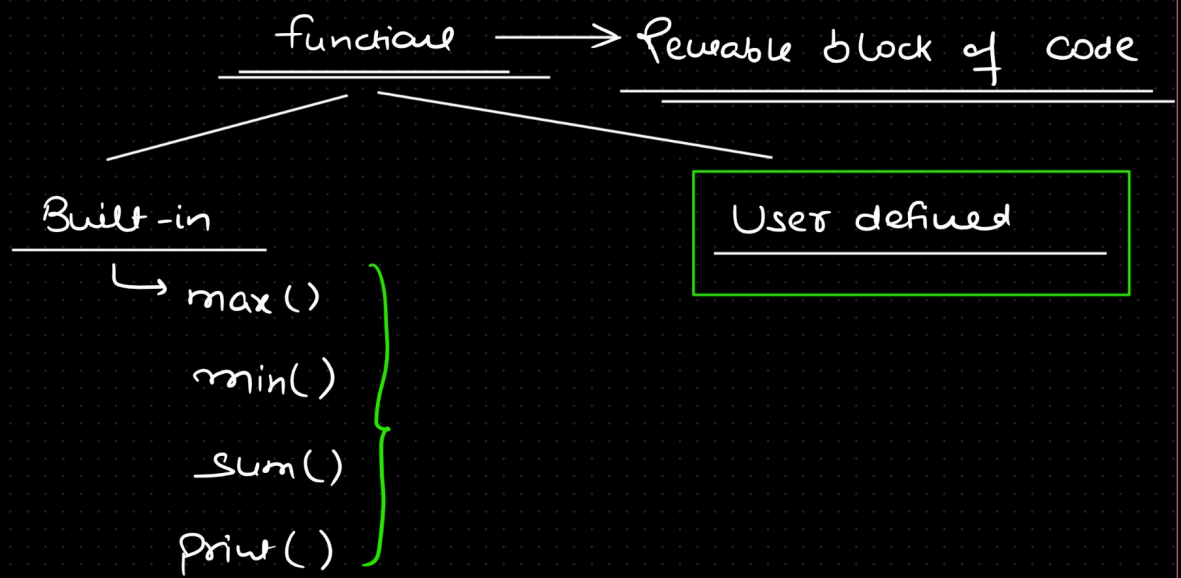
Count = 5

continue → skip

```
i = 1 4
while i < 3:
    j = 0
    while j < 5:
        j = j + 1
        if j == 3:
            continue
        print(j, end = " ")
    i = i + 1
```

i = 1	i = 2	i = 3
J = 0 1 2 3 4 5 3 4 5	J = 0 1 2 3 4 5	
1 2 4 5	1 2 4 5	

part → placeholder for
writing the
future code



Armstrong Numbers

↳ num = 153

$$\underline{\underline{1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153}}$$

num = 370

$$\underline{\underline{3^3 + 7^3 + 0^3 = 370}}$$

→ is_armstrong(num):

↳ True / False

function definition

```
def is_armstrong(num):
```

```
    str_num = str(num)
```

```
    num_digits = len(str_num)
```

```
    sum_of_powers = sum(int(digit)**num_digits for digit in str_num)
```

```
    return sum_of_powers == num
```

function calling

```
result = is_armstrong(150)
```

```
print(result)
```

built-in
exponentiation (power)

num = 153

str_num → "153"

num_digits → 3

True/false

$$1^3 + 5^3 + 3^3 = 153 = \text{sum}$$

of powers

Nested function

```
def calculator(x, y):
```

```
    def add(a, b):
```

```
        return a + b
```

```
    def multiply(a, b):
```

```
        return a * b
```

```
    def calc(op, a, b):
```

```
        if op == 'add':
```

```
            return add(a, b)
```

```
        else:
```

```
            return multiply(a, b)
```

```
    return calc('add', x, y), calc('multiply', x, y)
```

```
result = calculator(2, 3)
```

```
print(result)
```

function calling

(5, 6)